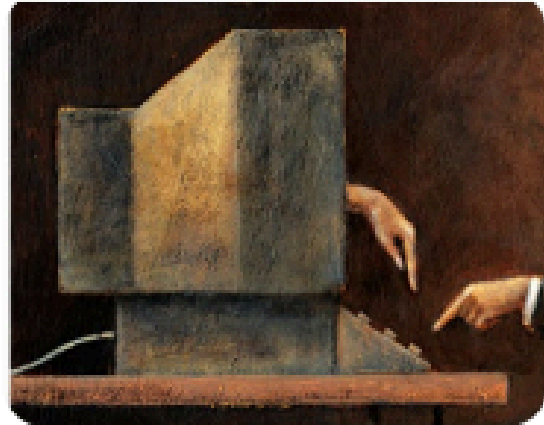
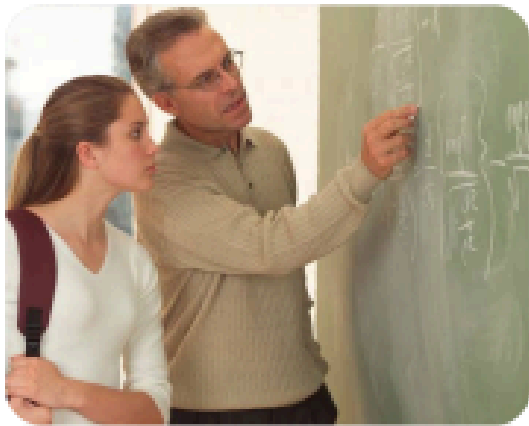




GRADO

PROGRAMACIÓN Y ESTRUCTURAS DE DATOS AVANZADAS

ENUNCIADO PRÁCTICA 2

Curso 2025-2026

Alejandro Fernández Polo
78153714J

afernande7115@alumno.uned.es

ÍNDICE

Cuestiones Teóricas	2
1. Indica y razona sobre el coste temporal y espacial del algoritmo.....	2
2. Describe alternativas al esquema utilizado, si las hay, y compara su coste con el de la solución realizada.....	2
Ejemplos de ejecución	3
Ejemplo 1:.....	3
Ejemplo 2:.....	3
Ejemplo 3:.....	4
Ejemplo 4:.....	4
Ejemplo 5:.....	4
Código fuente	5
Clase Main:.....	5
Clase Archives:.....	7
Clase Help:.....	10
Clase Course:.....	10
Clase Restrictions:.....	11
Clase Backtracking:.....	12

IMPORTANTE: El nombre del ejecutable es: Alejandro_Fernandez_Polo_PEC2.jar

Cuestiones Teóricas

1. Indica y razona sobre el coste temporal y espacial del algoritmo.

Coste Temporal

El coste temporal en el peor de los casos es factorial cuadrático, de orden $O((n!)^2)$.

- El algoritmo debe asignar a cada curso un par de recursos, Aula y Profesor.
- En el primer curso existen n aulas disponibles y n profesores disponibles, es decir, $n \times n = n^2$ combinaciones posibles.
- En el segundo curso quedan $n - 1$ aulas y $n - 1$ profesores, por lo tanto, $(n - 1)^2$ combinaciones.
- En el curso k , tenemos $(n - k + 1)^2$ combinaciones.

Lo que nos lleva a la conclusión inicial, el coste es de orden $O((n!)^2)$.

Coste Espacial

El coste espacial del algoritmo es lineal, de orden $O(n)$.

Lo obtenemos debido a los recursos de memoria necesarios para mantener el estado de ejecución en cada instante.

- Estructuras de datos auxiliares:
 - solution: Array que almacena los objetos Course, tamaño n .
 - asigAula: Array de booleanos dedicado al control de aulas ocupadas, tamaño n .
 - asigProf: Array de booleanos dedicado al control de profesores ocupados, tamaño n .
- Pila de recursión: Ya que el algoritmo es recursivo y avanza curso a curso, la profundidad máxima de la pila de llamadas será de n niveles en el peor de los casos.

Visto esto sabemos que el consumo de memoria crece linealmente con la entrada, $O(n)$.

2. Describe alternativas al esquema utilizado, si las hay, y compara su coste con el de la solución realizada.

Existen dos alternativas principales el Algoritmo Voraz y el Esquema de Ramificación y Poda

- **Algoritmo Voraz**

En términos de tiempo de ejecución esta es la alternativa más eficiente. Este enfoque trataría de construir la solución paso a paso, tomando la decisión válida en cada instante, sin corregirla nunca.

El algoritmo recorrería la lista de cursos, asignando el primer aula y profesor que cumplan las restricciones en ese momento.

Su coste temporal sería polinómico, de $O(n^2)$. Esto es una grandísima mejora a comparación del coste del algoritmo de Vuelta Atrás, que es de $O((n!)^2)$.

El problema de este algoritmo es que no garantiza encontrar una solución aunque exista. Ya que podría asignar al curso 1 un profesor o aula solo válida para el curso n. Por lo tanto es mucho más recomendable usar un algoritmo que nos garantiza tener una solución, a pesar de un coste temporal más elevado.

- **Ramificación y Poda**

Esta alternativa es una variación del algoritmo de Vuelta Atrás, utilizada para encontrar una solución óptima, por ejemplo, si se quisiese minimizar la distancia entre aulas o maximizar la preferencia de los profesores.

Al igual que la Vuelta Atrás este algoritmo explora el árbol de soluciones. Sin embargo, en cada nodo calcula una cota del coste de la solución. Si la solución indica que esa rama no mejorará la solución ya disponible, se poda esa rama y se deja de explorar.

En el peor caso, su coste temporal sigue siendo el mismo que el de la Vuelta Atrás, $O((n!)^2)$, debido a que puede tener que explorar todo el árbol.

En el problema actual, dado que el enunciado nos pide encontrar cualquier asignación válida, y no hay una solución mejor que otra, este algoritmo carece de sentido, ya que no tenemos una medida para calcular la cota.

Ejemplos de ejecución

Estos ejemplos están generados usando una semilla fija en la generación de restricciones, para replicarlas hay que cambiar el constructor de la clase Restrictions, actualmente con una semilla aleatoria, a este:

```
public Restrictions() {  
    this.rnd = new Random(12);  
}
```

Ejemplo 1:

Entrada:	Salida:
1	0

Ejemplo 2:

Entrada:	Salida:
4	3 1 1 4 2 2 1 3 4 2 4 3

Ejemplo 3:

Entrada:	Salida:
10	1 1 6 3 2 7 2 3 1 4 4 2 5 5 3 6 6 8 9 7 4 8 8 5 7 9 9 10 10 10

Ejemplo 4:

Entrada:	Salida:
20	1 1 6 3 2 7 2 3 1 4 4 2 5 5 3 6 6 4 8 7 9 7 8 5 12 9 14 9 10 8 10 11 12 14 12 10 11 13 11 18 14 13 15 15 15 13 16 18 20 17 19 17 18 17 19 19 20 16 20 16

Ejemplo 5:

Entrada:	Salida:
30	1 1 6 3 2 7 2 3 1 4 4 2 5 5 3 6 6 4 8 7 9 7 8 5 12 9 14 9 10 8 10 11 12 14 12 10 11 13 11 18 14 13 15 15 15 13 16 18 17 17 16 19 18 19 16 19 23 20 20 20 22 21 17 21 22 22 23 23 21 24 24 25 26 25 27 29 26 24 28 27 26 27 28 28 30 29 29 25 30 30

Código fuente

Clase Main:

```
public class Main {  
  
    public static void main(String[] args) {  
        // Instancia del helper de ayuda  
        Help help = new Help();  
        // Variable para manejar archivos, inicialmente null
```

```

Archives a = null;

// Validación: si hay 4 argumentos, verifica si alguno
solicita ayuda
    if (args.length == 4) {
        if (args[0].equalsIgnoreCase("-h") ||
args[1].equalsIgnoreCase("-h")) {
            help.showHelp();
        } else {
            // Número de argumentos inválido
            System.out.println("Error de argumentos, vea ayuda
con [-h]");
        }
    }

// Caso 1: Procesamiento estándar sin flags
// El primer argumento es directamente el archivo de
entrada
    else if (!args[0].equalsIgnoreCase("-h") &&
!args[0].equalsIgnoreCase("-t")) {
        a = new Archives(args[0]);
        a.loadBacktracking(false); // Ejecuta backtracking sin
mostrar trazas

        if (args.length == 1) {
            // Solo archivo de entrada: muestra resultado en
consola
            a.getSolution(null);
        } else {
            // Con archivo de salida: exporta resultado al
archivo
            a.getSolution(args[1]);
        }
    }

// Caso 2: Flag de ayuda
    else if (args[0].equalsIgnoreCase("-h")) {
        help.showHelp();
    }

// Caso 3: Modo trazas activado
// Muestra el proceso paso a paso del backtracking
    else if (args[0].equalsIgnoreCase("-t")) {
        a = new Archives(args[1]); // El archivo está en la
segunda posición
        a.loadBacktracking(true); // Ejecuta backtracking
mostrando trazas

        if (args.length == 2) {
            // Solo flag y archivo: muestra resultado en
consola

```

```

        a.getSolution(null);
    } else {
        // Con archivo de salida: exporta resultado al
archivo
        a.getSolution(args[2]);
    }
}
// Caso por defecto: formato no válido
else {
    System.out.println("Error de argumentos, vea ayuda con
[-h]");
}
}
}

```

Clase Archives:

```

import java.io.BufferedWriter;
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

/**
 * Clase de gestión de archivos para el sistema de backtracking.
 * Responsable de:
 * - Leer la configuración inicial desde un archivo de entrada
 * - Ejecutar el algoritmo de backtracking
 * - Exportar la solución a consola o archivo de salida
 */
public class Archives {
    /** Ruta o nombre del fichero de entrada que contiene el número
de cursos. */
    public String fichero;
    /** Instancia del motor de backtracking que resuelve el problema.
*/
    public Backtracking backtracking;

    /**
     * Constructor que asocia la instancia a un archivo de entrada.
     *
     * @param fichero nombre del fichero a leer (ruta relativa o
absoluta)
     */
    public Archives(String fichero) {
        this.fichero = fichero;
    }
}

```



```

    }

    /**
     * Carga y ejecuta el algoritmo de backtracking desde el archivo
de entrada.
     * Lee el número de cursos del archivo y lanza el proceso de
asignación.
     *
     * Formato esperado del archivo de entrada:
     * Primera línea: número entero N (cantidad de cursos, aulas y
profesores)
     *
     * @param show si true, muestra trazas detalladas del proceso de
búsqueda
     */
    public void loadBacktracking(Boolean show) {
        try {
            // Abre el fichero y prepara el lector de enteros
            Scanner sr = new Scanner(new File(fichero));
            // Inicializa el backtracking con el número leído del
archivo
            this.backtracking = new Backtracking(sr.nextInt());

            if (show) {
                System.out.println("Se va a intentar asignar " +
backtracking.getNumberOfCourses() + " cursos, " +
                    backtracking.getNumberOfCourses() + " aulas
y " + backtracking.getNumberOfCourses() + " profesores.");
            }
            // Ejecuta el algoritmo de backtracking comenzando por
el curso 1
            backtracking.cursosEscuela(1, show);
        } catch (Exception e) {
            // Si ocurre cualquier error (fichero inexistente,
formato inválido, etc.)
            System.out.println("Error, el fichero no existe.");
        }
    }

    /**
     * Obtiene y muestra la solución del backtracking.
     * Puede escribir en consola o exportar a un archivo según el
parámetro.
     *
     * Formatos de salida:
     * - Si hay solución: líneas con formato "aula curso profesor"
     * - Sin solución: imprime "0"
     *

```

```

    * @param fichero ruta del archivo de salida. Si es null, muestra
por consola
    */
    public void getSolution(String fichero) {
        // Salida por consola cuando no se especifica fichero de
salida
        if (fichero == null) {
            for (Course course : backtracking.getSolution()) {
                // Si algún curso no tiene asignación, no hay
solución completa
                if (course == null) {
                    System.out.println("0");
                    return;
                }
                System.out.println(course);
            }
            return;
        }

        // Verifica que el fichero de salida no exista para evitar
sobreescritura accidental
        if (new File(fichero).exists()) {
            System.err.println("Error: El fichero de salida ya
existe.");
            return;
        }

        // Salida a fichero cuando se especifica ruta/nombre de
salida
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(fichero))) {
            for (Course course : backtracking.getSolution()) {
                // Si no hay solución válida, escribe "0"
                if (course == null) {
                    writer.write("0");
                    writer.newLine();
                    return;
                }
                // Escribe cada curso en formato "aula curso
profesor"
                writer.write(String.valueOf(course));
                writer.newLine();
            }
            System.out.println("Archivo creado correctamente.");
        } catch (IOException e) {
            // Manejo de errores de escritura
            System.err.println("Error en la escritura de archivo: "
+ e.getMessage());

```

```

    }
}

```

Clase Help:

```

public class Help {
    /**
     * Constructor clase Ayuda
     */
    public Help() {
    }

    /**Método que muestra por pantalla la ayuda*/
    public void showHelp() {
        System.out.println("Ayuda para el algoritmo de Asignación
de Cursos");
        System.out.println("La sintaxis del comando para la
Asignación de Cursos es:");
        System.out.println("[-t] [-h] [fichero_entrada]
[fichero_salida]");
        System.out.println("-h: Muestra esta ayuda. (OPCIONAL)");
        System.out.println("-t: Modo traza. Muestra mensajes
explicativos del funcionamiento del programa. (OPCIONAL)");
        System.out.println("fichero_entrada: Fichero de texto que
contiene la descripción de la asignación. (REQUERIDO)");
        System.out.println("fichero_salida: Fichero de texto donde
se guardará la solución. (OPCIONAL)");
        System.out.println("Si no se especifica fichero_salida, la
solución se mostrará por pantalla.");
    }
}

```

Clase Course:

```

/**
 * Representa un curso que necesita ser asignado a un aula y un
profesor.
 */
public class Course {
    // Identificador del aula asignada al curso
    private int classroom;
    // Identificador del profesor asignado al curso
    private int professor;
    // Número identificador único del curso
    private int number;
}

```

```

    /**
     * Constructor que inicializa un curso con su número
    identificador.
     *
     * @param number identificador del curso
     */
    public Course(int number) {
        this.number = number;
    }

    public void setProfessor(int professor) {
        this.professor = professor;
    }

    public void setClassroom(int classroom) {
        this.classroom = classroom;
    }

    /**
     * Representación en formato de texto del curso.
     * Formato: "aula curso profesor"
     *
     * @return cadena con los valores de aula, número de curso y
    profesor separados por espacios
     */
    @Override
    public String toString() {
        return classroom + " " + number + " " + professor;
    }
}

```

Clase Restrictions:

```

import java.util.Random;

/**
 * Gestiona las restricciones de asignación entre cursos, aulas y
    profesores.
 * Utiliza generación aleatoria para simular reglas de validación.
 */
public class Restrictions {

    // Generador de números aleatorios para simular decisiones de
    validación
    private Random rnd;
}

```

```

/**
 * Constructor que inicializa el generador aleatorio.
 */
public Restrictions() {
    this.rnd = new Random(12);
}

/**
 * Verifica si un curso puede ser asignado a un aula específica.
 * Simula restricciones de compatibilidad aula-curso.
 *
 * @param classroom identificador del aula
 * @param course identificador del curso
 * @return true si la asignación es válida, false en caso
contrario (aleatorio)
 */
public boolean valid(int classroom, int course) {
    // Devuelve verdadero o falso aleatoriamente
    return rnd.nextBoolean();
}

/**
 * Verifica si un profesor tiene la especialidad requerida para
impartir un curso.
 * Simula restricciones de compatibilidad profesor-curso.
 *
 * @param professor identificador del profesor
 * @param course identificador del curso
 * @return true si el profesor puede impartir el curso, false en
caso contrario (aleatorio)
 */
public boolean specialty(int professor, int course) {
    // Devuelve verdadero o falso aleatoriamente
    return rnd.nextBoolean();
}
}

```

Clase Backtracking:

```

/**
 * Implementa un algoritmo de backtracking para asignar cursos a
aulas y profesores.
 * Resuelve el problema de asignación de recursos con restricciones
de:
 * - Un aula por curso
 * - Un profesor por curso
 * - Validación de capacidad de aulas
 * - Validación de especialidad de profesores

```

```

*/
public class Backtracking {

    // Gestor de restricciones de validación
    private Restrictions restrictions;
    // Array que almacena la solución: cada posición contiene un
curso asignado
    private Course[] solution;
    // Indica si se ha encontrado una solución válida
    private boolean success;
    // Control de aulas ya asignadas (evita duplicados)
    private boolean[] asigAula;
    // Control de profesores ya asignados (evita duplicados)
    private boolean[] asigProf;
    // Número total de cursos a asignar
    private int numberOfCourses;

    /**
     * Constructor que inicializa el sistema de backtracking.
     *
     * @param n número de cursos, aulas y profesores (mismo tamaño)
     */
    public Backtracking(int n) {
        this.restrictions = new Restrictions();
        this.solution = new Course[n];
        this.success = false;
        this.asigAula = new boolean[n];
        this.asigProf = new boolean[n];
        this.numberOfCourses = n;
    }

    public int getNumberOfCourses() {
        return numberOfCourses;
    }

    public Course[] getSolution() {
        return solution;
    }

    /**
     * Algoritmo recursivo de backtracking para asignar cursos.
     * Prueba todas las combinaciones posibles de aula-profesor para
cada curso
     * hasta encontrar una solución válida o agotar todas las
posibilidades.
     *
     * @param courseNumber número del curso actual a asignar
(1-indexed)

```

```

    * @param show si true, muestra trazas del proceso de búsqueda
    */
    public void cursosEscuela(int courseNumber, boolean show) {

        // Caso base: ya se encontró solución, terminar
        if (this.success) return;

        if (show) {
            System.out.println("Intentando asignar curso " +
courseNumber);
        }

        // Crear objeto curso para asignación
        Course course = new Course(courseNumber);
        int classroom = 1;

        // BUCLE EXTERNO: probar todas las aulas disponibles
        while (classroom <= this.numberOfCourses && !this.success)
{
            // Verificar si el aula cumple las restricciones del
curso
            boolean validClassroom =
this.restrictions.valid(classroom, courseNumber);

            if (show) {
                System.out.println("Probando aula " + classroom + "
para el curso " + courseNumber + ". ¿Aula válida? " +
validClassroom);
            }

            // Si el aula está libre Y es válida para el curso
            if (!this.asigAula[classroom-1] && validClassroom) {
                int professor = 1;

                // BUCLE INTERNO: probar todos los profesores
disponibles
                while (professor <= this.numberOfCourses &&
!this.success) {
                    // Verificar si el profesor tiene la
especialidad requerida
                    boolean validProfessor =
this.restrictions.specialty(professor, courseNumber);

                    if (show) {
                        System.out.println("Probando profesor " +
professor + " para el curso " + courseNumber + ". ¿Profesor
válido? " + validProfessor);
                    }
                }
            }
        }
    }

```

```

        // Si el profesor está libre Y tiene la
especialidad
        if (!this.asigProf[professor-1] &&
validProfessor) {

            // *** MARCAR (ASIGNAR) ***
            course.setClassroom(classroom);
            course.setProfessor(professor);
            this.solution[courseNumber-1] = course;

            if (show) {
                System.out.println("Aqui se ha asignado
en la solucion el curso: " + this.solution[courseNumber-1]);
            }

            // Marcar recursos como ocupados
            this.asigAula[classroom-1] = true;
            this.asigProf[professor-1] = true;

            // CASO BASE: ¿Es el último curso?
            if (courseNumber == this.numberOfCourses) {
                this.success = true; // Solución
completa encontrada
            } else {
                // RECURSIÓN: intentar asignar el
siguiente curso
                cursosEscuela(courseNumber + 1, show);
            }

            // *** DESMARCAR (BACKTRACKING) ***
            // Si la recursión no tuvo éxito, deshacer
la asignación
            if (!this.success) {
                if (show) {
                    System.out.println("Deshaciendo
asignación del curso " + courseNumber + " (aula " + classroom + ",
profesor " + professor + ")");
                }
                // Liberar recursos
                this.asigAula[classroom-1] = false;
                this.asigProf[professor-1] = false;
                this.solution[courseNumber-1] = null;
            }
        }
        professor++; // Probar siguiente profesor
    }
}

```



```
        classroom++; // Probar siguiente aula
    }
}
```